

Understanding SQL Joins

Lecture 29

Innocent Nyalala

IIT Madras Zanzibar

Monday, June 9, 2025

Goal: Combine data from two or more tables based on related columns.

The Basics: What are Joins?

Imagine you have separate lists of information:

- **List A:** Your friends' names and their favorite colors.
- **List B:** Your friends' names and their favorite foods.

You want to create a *new* list that shows each friend's name, their favorite color, *and* their favorite food. Joins help you do exactly that with database tables!

Key Join Types (The Most Common Ones)

Scenario: University Data

We have two tables: **Students Table:**

student_id	student_name	department_id
1	Alice	101
2	Bob	102
3	Charlie	NULL
4	David	101

Departments Table:

department_id	department_name
101	Computer Science
102	Electrical Engineering
103	Mathematics

1. INNER JOIN (The "Only Matches" Join)

- **What it does:** Returns only the rows where there is a match in *both* tables. If a student doesn't have a matching department, or a department has no students, they won't show up.
- **Analogy:** You're looking for friends who have *both* a favorite sport AND a favorite movie listed. If they only have one, or neither, they're not on your combined list.

Use Case

Fetch students and their enrolled courses.

INNER JOIN - Syntax and Example

Syntax

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.matching_column = table2.matching_column;
```

Example (University Data): Show students and their assigned departments

```
SELECT
    S.student_name,
    D.department_name
FROM
    Students S
INNER JOIN
    Departments D ON S.department_id = D.department_id;
```

Result:

student_name	department_name
Alice	Computer Science

2. LEFT JOIN (or LEFT OUTER JOIN)

- **What it does:** Returns all rows from the *left* table, and the matching rows from the *right* table. If there's no match in the right table, NULL values will appear for the right table's columns.
- **Analogy:** You want to list *all* your friends from Notebook 1 (Favorite Sports), and if they also have a favorite movie in Notebook 2, you'll add that too. If not, you just list their sport and leave the movie blank.

Syntax and Example

Syntax:

```
SELECT columns  
FROM table1 -- This is your LEFT table  
LEFT JOIN table2 -- This is your RIGHT table  
ON table1.matching_column = table2.matching_column;
```

Example (University Data): Show all students and their departments (if they have one).

```
SELECT  
    S.student_name,
```

3. RIGHT JOIN (or RIGHT OUTER JOIN)

- **What it does:** Returns all rows from the *right* table, and the matching rows from the *left* table. If there's no match in the left table, NULL values will appear for the left table's columns.
- **Analogy:** You want to list *all* the favorite movies from Notebook 2, and if a friend who likes that movie is also in Notebook 1 (Favorite Sports), you'll add their sport too. If not, you just list the movie and leave the sport blank.

Syntax and Example

Syntax:

```
SELECT columns  
FROM table1 --- This is your LEFT table  
RIGHT JOIN table2 --- This is your RIGHT table  
ON table1.matching_column = table2.matching_column;
```

Example (University Data): Show all departments and the students assigned to them (if any).

```
SELECT  
    S.student_name,
```

4. FULL OUTER JOIN (The "Everything from Both" Join)

- **What it does:** Returns all rows when there is a match in one of the tables. It combines the results of both LEFT JOIN and RIGHT JOIN.
- **Analogy:** You want *every single friend* from both Notebook 1 and Notebook 2, and for each friend, you want to see their favorite sport (if they have one) and their favorite movie (if they have one). If they only have one, the other field is blank.

MySQL Workaround (Example)

```
-- This is a common way to simulate FULL OUTER JOIN in MySQL
SELECT
    S.student_name,
    D.department_name
FROM
    Students S
LEFT JOIN
    Departments D ON S.department_id = D.department_id
UNION
SELECT
    S.student_name,
    D.department_name
FROM
    Students S
RIGHT JOIN
```


5. CROSS JOIN (The "Every Possible Combination" Join)

- **What it does:** Returns the Cartesian product of the two tables. This means every row from the first table is combined with every row from the second table. There's no ON clause because it doesn't look for matches.
- **Analogy:** You have 3 shirts and 2 pairs of pants. A cross join would show you every possible outfit combination (Shirt 1 + Pants 1, Shirt 1 + Pants 2, Shirt 2 + Pants 1, etc.).

Syntax and Example

Syntax:

```
SELECT columns  
FROM table1  
CROSS JOIN table2;
```

Example (University Data): (Often used for generating combinations or for performance testing, less for direct data matching)

```
SELECT  
    S.student_name,  
    D.department_name
```

- **Aliases:** Use short aliases (like S for Students and D for Departments) to make your queries shorter and easier to read (FROM Students S).
- **ON Clause:** This is where you specify the *relationship* between your tables (e.g., S.department_id = D.department_id).
- **WHERE Clause:** You can still use a WHERE clause after a join to filter the *joined* results further.
- **Understanding Your Data:** Before writing a join, always ask yourself: "What columns do these tables have in common, and what result do I want to see?"

In-Class Assignment/Quiz: Joins in SSMS (10 Marks)

Instructions: Work in pairs. You will be provided with a SQL Server Management Studio (SSMS) environment with two tables: Employees and Departments. Your task is to write SQL queries to answer the questions below.

Scenario: Imagine a company database with information about employees and the departments they belong to.

Table Structure and Sample Data

Employees Table:

Column Name	Data Type	Description
EmployeeID	INT (PK)	Unique ID for each employee
FirstName	VARCHAR(50)	Employee's first name
LastName	VARCHAR(50)	Employee's last name
DepartmentID	INT (FK)	ID of the department (can be NULL)
Salary	DECIMAL(10, 2)	Employee's salary

Departments Table:

Column Name	Data Type	Description
DepartmentID	INT (PK)	Unique ID for each department
DepartmentName	VARCHAR(50)	Name of the department
Location	VARCHAR(50)	City where the department is located

Sample

Data (Create and Populate in SSMS):

Assignment Questions

- ❶ (2 Marks) INNER JOIN: List the FirstName, LastName, and DepartmentName for all employees who are currently assigned to a department.

```
SELECT E.FirstName, E.LastName, D.DepartmentName
FROM Employees E
INNER JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

Expected Output:

FirstName	LastName	DepartmentName
John	Doe	Sales
Jane	Smith	Marketing
Peter	Jones	Sales
Chris	Davis	Engineering
Anna	White	Sales

- ❷ (2 Marks) LEFT JOIN: Show the FirstName, LastName, and DepartmentName for *all* employees. If an employee is not assigned to a department, their DepartmentName should appear as NULL.

```
SELECT E.FirstName, E.LastName, D.DepartmentName
FROM Employees E
LEFT JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

FirstName	LastName	DepartmentName
John	Doe	Sales
Jane	Smith	Marketing

Assignment Questions (Cont.)

- ③ (2 Marks) **RIGHT JOIN:** Display the DepartmentName and the FirstName of any employees associated with that department. Include departments that currently have no employees.

```
SELECT D.DepartmentName, E.FirstName  
FROM Employees E  
RIGHT JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

Expected Output:

DepartmentName	FirstName
Sales	John
Sales	Peter
Sales	Anna
Marketing	Jane
Engineering	Chris
Human Resources	NULL

- ④ (2 Marks) **INNER JOIN with WHERE Clause:** Find the FirstName, LastName, and DepartmentName of all employees who earn more than \$70,000 and are in a department.

```
SELECT E.FirstName, E.LastName, D.DepartmentName  
FROM Employees E  
INNER JOIN Departments D ON E.DepartmentID = D.DepartmentID  
WHERE E.Salary > 70000.00;
```

Assignment Questions (Cont.)

- 5 (2 Marks) CROSS JOIN (Careful with this one!): Show *all possible combinations* of FirstName and DepartmentName. (This query will produce many rows, so be aware!)

```
SELECT E.FirstName, D.DepartmentName  
FROM Employees E  
CROSS JOIN Departments D;
```

Expected Output (partial):

FirstName	DepartmentName
John	Sales
John	Marketing
John	Engineering
John	Human Resources
Jane	Sales
Jane	Marketing
...	...

Instructions for Submission:

- For each question, write the SQL query in SSMS.
- Take a screenshot of your query and its result in SSMS for each question.